

b) Scraping completo de productos usando el sitemap

Objetivo: descargar la información completa de **todos** los productos de la tienda virtual (`http://localhost:3000`) y guardarla en `data/productos.csv` .

Estrategia:

1. Leer el sitemap índice (`/sitemap.xml`) para descubrir el sitemap específico de productos.
2. Leer `sitemap-productos.xml` , que expone la URL de detalle de **cada** producto (`/productos/{id}`) — a diferencia de clientes, el sitemap de productos sí enumera cada elemento individualmente, por lo que no se necesita paginación.
3. Visitar cada URL de producto y extraer sus datos con BeautifulSoup, usando los atributos `data-*` y las propiedades `itemprop` (microdatos `schema.org/Product`) que ya expone la página.

```
import csv
import os
import re
import requests
from bs4 import BeautifulSoup

BASE_URL = "http://localhost:3000"
HEADERS = {"User-Agent": "MineriaWeb-2026-2/1.0 (+scraper-tienda-virtual)"}

def get_soup(url: str, parser: str = "html.parser") -> BeautifulSoup:
    respuesta = requests.get(url, headers=HEADERS, timeout=10)
    respuesta.raise_for_status()
    return BeautifulSoup(respuesta.content, parser)
```

1. Descubrir el sitemap de productos a partir del sitemap índice

```
sitemap_index = get_soup(f"{BASE_URL}/sitemap.xml", "xml")
sub_sitemaps = [loc.text for loc in sitemap_index.find_all("loc")]
print("Sub-sitemaps encontrados:")
for url in sub_sitemaps:
    print(" -", url)

productos_sitemap_url = next(url for url in sub_sitemaps if "sitemap-productos" in url)
print("\nSitemap de productos:", productos_sitemap_url)
```

Sub-sitemaps encontrados:

- <http://localhost:3000/sitemap-general.xml>
- <http://localhost:3000/sitemap-productos.xml>
- <http://localhost:3000/sitemap-clientes.xml>
- <http://localhost:3000/sitemap-ordenes.xml>
- <http://localhost:3000/sitemap-testimonios.xml>

Sitemap de productos: <http://localhost:3000/sitemap-productos.xml>

2. Extraer las URLs de cada producto

El sitemap de productos también incluye `/productos` y `/productos/nuevo` (páginas de listado y de creación), así que filtramos únicamente las URLs con formato `/productos/{id numérico}`.

```
sitemap_productos = get_soup(productos_sitemap_url, "xml")
producto_urls = [
    loc.text for loc in sitemap_productos.find_all("loc") if
    re.search(r"/productos/\d+$", loc.text)
]
print(f"Total de productos encontrados en el sitemap: {len(producto_urls)}")
print(producto_urls[:5])
```

```
Total de productos encontrados en el sitemap: 90
['http://localhost:3000/productos/1', 'http://localhost:3000/productos/2',
'http://localhost:3000/productos/3', 'http://localhost:3000/productos/4',
'http://localhost:3000/productos/5']
```

3. Scraping del detalle de cada producto

Cada página `/productos/{id}` renderiza un `<section itemscope itemtype="https://schema.org/Product">` con atributos `data-*` (id, código, categoría, subcategoría, precio, stock) y microdatos `itemprop` para nombre, descripción y la calificación agregada de reseñas.

```
def scrape_producto(url: str) -> dict:
    soup = get_soup(url)
    section = soup.select_one("section[data-producto-id]")

    nombre = section.select_one("h2[itemprop=name]").get_text(strip=True)
    descripcion = section.select_one("p[itemprop=description]").get_text(strip=True)

    rating_el = section.select_one("[itemprop=aggregateRating]")
    if rating_el:
        calificacion_promedio = rating_el.select_one("[itemprop=ratingValue]").get_text(strip=True)
        total_resenas = rating_el.select_one("[itemprop=reviewCount]").get_text(strip=True)
    else:
        calificacion_promedio, total_resenas = "", "0"

    return {
        "id": section["data-producto-id"],
        "codigo": section["data-producto-codigo"],
        "nombre": nombre,
        "descripcion": descripcion,
        "categoria": section["data-categoria"],
```

```

        "subcategoria": section["data-subcategoria"],
        "precio": section["data-precio"],
        "stock": section["data-stock"],
        "calificacion_promedio": calificacion_promedio,
        "total_resenas": total_resenas,
        "url": url,
    }

```

```

productos = []
for url in producto_urls:
    try:
        productos.append(scrape_producto(url))
    except Exception as error:
        print(f"Error al procesar {url}: {error}")

print(f"Productos scrapeados correctamente: {len(productos)}")
productos[:2]

```

Productos scrapeados correctamente: 90

```

[{'id': '1',
  'codigo': 'PROD-001',
  'nombre': 'Apple iPhone 15 Pro Max 256GB',
  'descripcion': 'Smartphone premium con chip A17 Pro fabricado en 3nm, camara triple de 48MP con teleobjetivo periscopico de 5x y pantalla Super Retina XDR de 6.7 pulgadas con ProMotion a 120Hz. Cuenta con marco de titanio, Dynamic Island, puerto USB-C y resistencia al agua IP68.',
  'categoria': 'Smartphones',
  'subcategoria': 'Gama Alta',
  'precio': '5599',
  'stock': '18',
  'calificacion_promedio': '3.5',
  'total_resenas': '4',
  'url': 'http://localhost:3000/productos/1'},
 {'id': '2',
  'codigo': 'PROD-002',
  'nombre': 'Apple iPhone 15 128GB',
  'descripcion': 'Smartphone con chip A16 Bionic, camara dual de 48MP con modo retrato mejorado y Dynamic Island para notificaciones interactivas. Incluye pantalla Super Retina XDR de 6.1 pulgadas, puerto USB-C y proteccion Ceramic Shield.',
  'categoria': 'Smartphones',
  'subcategoria': 'Gama Alta',
  'precio': '3799',
  'stock': '25',
  'calificacion_promedio': '3.3',
  'total_resenas': '4',
  'url': 'http://localhost:3000/productos/2'}]

```

4. Guardar los datos en data/productos.csv

```

DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
OUTPUT_PATH = os.path.join(DATA_DIR, "productos.csv")

with open(OUTPUT_PATH, mode="w", newline="", encoding="utf-8") as archivo:
    writer = csv.DictWriter(archivo, fieldnames=productos[0].keys())
    writer.writeheader()
    writer.writerows(productos)

```

```
print(f"Se guardaron {len(productos)} productos en {OUTPUT_PATH}")
```

Se guardaron 90 productos en /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-02/notebooks/./data/productos.csv